

FINAL PROJECT 1

AI Customer Complaint & Case Processing System

A GenAI batch workflow that reads complaint documents and produces structured data, customer emails and management summaries

Python · OpenAI · LangChain · LangGraph · Pydantic

Ashish Jain

Certification Programme in Generative and Agentic AI Development

Reading complaints by hand does not scale

What a human does today

- Read the document — a PDF, a Word form or a transcript
- Type the customer and case details into a system
- Decide whether the case needs escalation
- Write a reply to the customer
- Write a summary note for the manager

Why it is a poor fit for a human, and a good fit for an LLM

1

Repetitive

The same five steps for every single complaint that arrives.

2

Inconsistent

Two humans categorise and escalate the same case differently.

3

Unstructured input

Nothing is in a fixed position, so a rule-based parser cannot work.

One command, three artefacts per document

Batch : `python app.py`

reads every .pdf, .docx and .txt file in the data/ folder

Web : `streamlit run streamlit_app.py`

The same pipeline in a browser — this is what gets deployed

1

Structured case data

Customer name, email, phone, category, issue, resolution, three Yes/No business flags, case status and priority — validated against a schema.

`structured_data/*.json`

2

Customer reply email

A professional response addressed to the customer, restating the issue and giving the resolution or the next step — inventing nothing.

`customer_emails/*.txt`

3

Internal case summary

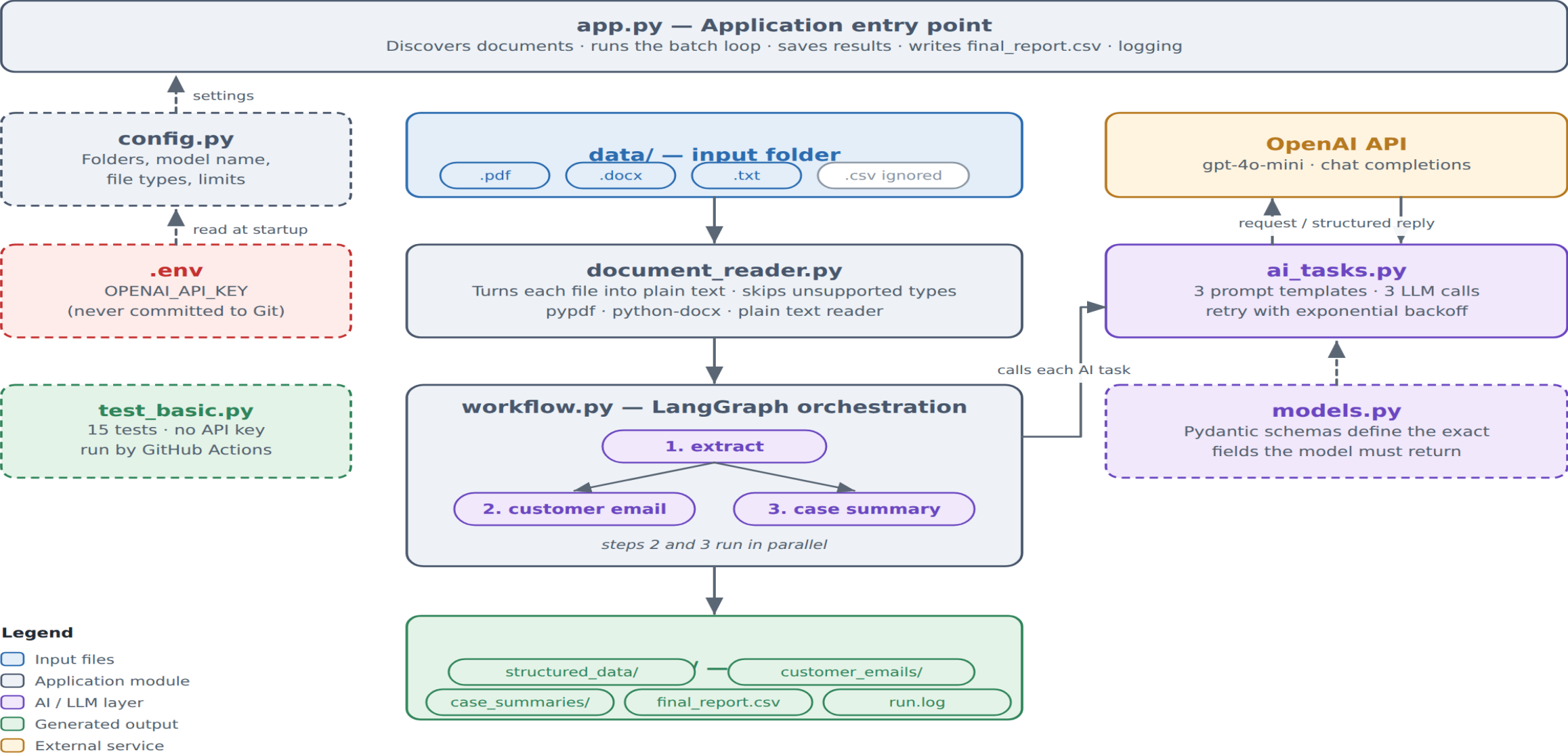
Overview, key issue, action taken, current status and one concrete recommended next action for the service manager.

`case_summaries/*.txt`

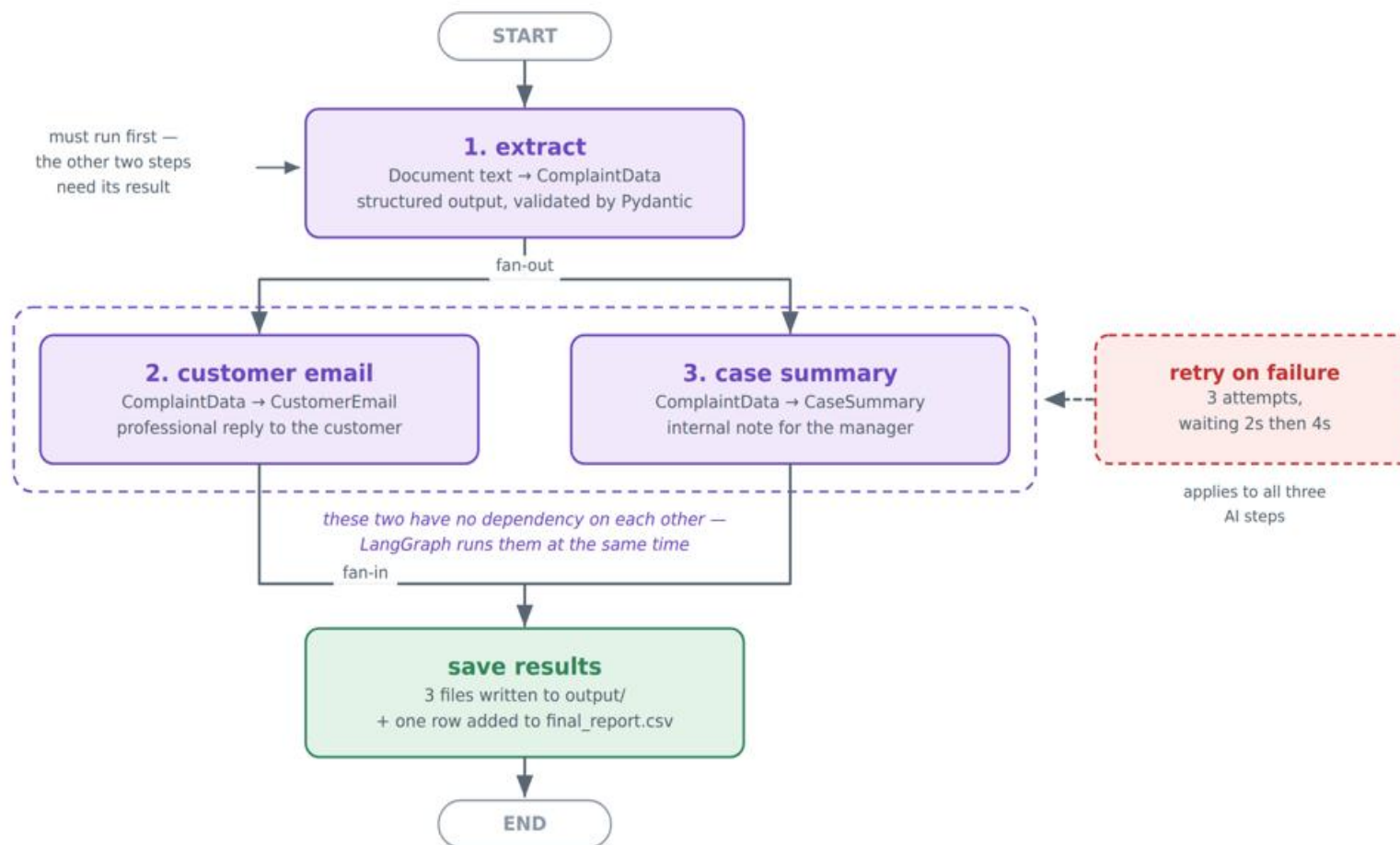
Plus one consolidated `final_report.csv` across the whole batch, and a timestamped `run.log`

System Architecture

AI Customer Complaint & Case Processing System



Sequential where it must be, parallel where it can be



Why a graph?

- Extraction must run first — both other tasks consume its output.
- The email and the summary depend only on that output, not on each other.
- Two edges out of one node, so LangGraph runs them in the same superstep.
- A fourth task is one node and one edge — the other three do not change.

The model fills in a schema — we never parse text

```
chain = EXTRACTION_PROMPT | llm.with_structured_output(ComplaintData)
result = chain.invoke({"document_text": text})

# result is a validated ComplaintData object, not a string
```

1

The schema is the prompt

Each `Field(description=...)` in `models.py` is sent to the model as part of the instruction.

2

Invalid answers cannot pass

`complaint_category` is a `Literal` of nine values. An invented category fails validation.

3

Absence is recordable

Optional fields default to `None`, so the model can say a detail is missing instead of inventing one.

4

The report is always clean

No text parsing anywhere in the project, so the CSV cannot be corrupted by odd wording.

Four measures that stop the model inventing facts

The shared rule, included in all three prompts

“Use ONLY information that is present in the text you are given. If something is not mentioned, leave it empty or false. Never invent names, phone numbers, dates, amounts or promises.”

Why it matters

An email that promises a refund the company never agreed to is worse than no email at all. So the email prompt adds: if no resolution is recorded, say the case is under review and give the next step — do not promise a refund or a date.

1

A rule in every prompt

Defined once as a constant and included in all three system messages.

2

Structured output

Fixed fields, not free prose in which an invented detail can hide.

3

Optional fields

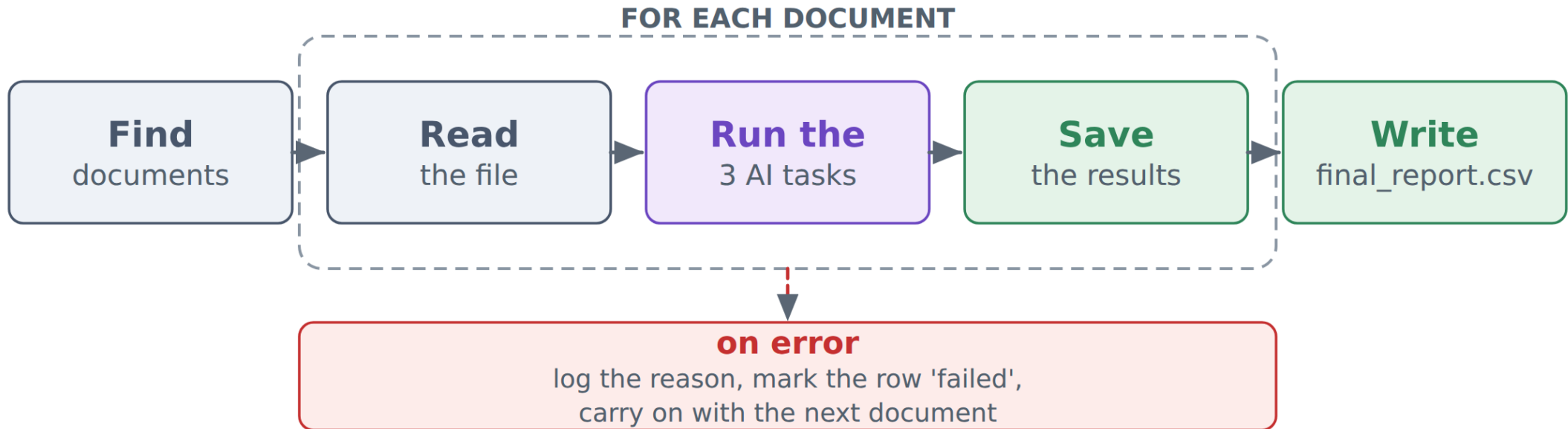
None is a legitimate answer, so the model has somewhere to put absence.

4

Generation reads the extraction

Not the raw document — so it can only use facts already validated.

One bad document never stops the batch



1 The API call

`call_with_retry()` makes up to three attempts, waiting 2 seconds and then 4. Recovers from dropped connections and rate limits.

2 The file

A corrupt PDF, a scanned image with no text layer, or an almost-empty file is rejected with a message the user can act on.

3 The document

A try/except per document. The failure is logged, the batch continues, and the document still gets a CSV row marked 'failed'.

Technology choices and why

OpenAI gpt-4o-mini

Strong structured-output support at low cost — about \$0.03 for the whole sample batch.

LangChain

Prompt templates and `with_structured_output`, which binds a Pydantic schema to the model.

LangGraph

Expresses the three tasks as a graph, giving real parallel execution of the two independent tasks.

Pydantic v2

Defines the required output shape and validates the reply before the application uses it.

pypdf · python-docx

Mature pure-Python readers, one per format, behind a single `read_document()` function.

pytest · GitHub Actions

15 tests that need no API key, run automatically on every push.

RESULTS

Six documents, three formats, one run

6

documents processed
from 3 file formats

18

API calls
(3 per document)

~\$0.03

cost of a complete
run on gpt-4o-mini

15

automated tests,
no API key needed

DOCUMENT	STATUS	CATEGORY	ESCALATE	CASE STATUS
complaint_001.pdf	success	Billing	No	Resolved
complaint_002.pdf	success	Delivery	Yes	Escalated
complaint_003.txt	success	Warranty	Yes	Escalated
complaint_004.docx	success	Account Access	Yes	Escalated
complaint_005.docx	success	Service Quality	No	Resolved
enquiry_006.txt	success	Other	No	Closed

What to notice

- The last row is an enquiry, not a complaint — the system sets is_complaint to No.
- Escalation comes from meaning, not keywords: one document mentions a manager but is resolved.
- An unsupported .csv in the folder is skipped without error.

Tested, versioned and reproducible

Testing

- 15 tests in test_basic.py
- None of them calls the API — free, and under a second
- Covers file reading, Pydantic validation and CSV building
- A test proves an invented category is rejected

Version control

- .gitignore excludes .env and output/
- The API key can never reach GitHub
- A commit-by-commit history, not one bulk commit
- .env.example is committed in its place

Continuous integration

- GitHub Actions runs pytest on every push
- No secret needs configuring, because the tests need no key
- A green tick on every commit
- Defined in .github/workflows/tests.yml

Where each assessed skill lives in the code

Python

all seven files

LLM API integration

`ai_tasks.build_llm()`

Prompt Engineering

three prompts, `ai_tasks.py`

Structured Outputs

`with_structured_output()`

Pydantic

`models.py` – three schemas

Batch Processing

`app.py` – the loop + CSV

Sequential / Parallel

`workflow.py` – LangGraph

Error Handling

`retry`, `validation`, `try/except`

Modular Code

six single-purpose modules

File Processing

`document_reader.py`

Git / GitHub

`.gitignore` + Actions

Basic Logging

screen + `output/run.log`

What it does not do yet, and what comes next

Current limitations

- Scanned PDFs have no text layer — they are correctly rejected, not silently misread.
- Documents are processed one at a time; the parallelism is within a document, not across them.
- No human review step — emails are written to disk, not sent.
- No confidence threshold at which the system would ask for human input.
- English only; other languages are untested.

Next steps

- 1 OCR**
Add Tesseract so scanned complaint documents can be processed too.
- 2 Concurrency across documents**
A thread pool — the program spends nearly all its time waiting on the API.
- 3 Automatic routing**
Send escalated cases to the responsible team using category and priority.
- 4 Policy retrieval (RAG)**
Ground the email in the real refund window and service-level targets.

IN SUMMARY

Unstructured documents in, governed business output out

Structured output

Binding every reply to a Pydantic schema removes text parsing from the system entirely.

Explicit orchestration

LangGraph makes the dependency between the three tasks visible, and runs two of them in parallel.

Failures stay visible

Three levels of error handling; a failed document still appears in the report with its reason.

Thank you — questions welcome